

Coding & Solution

Date	03 July 2026
Team ID	xxxxxx
Project Name	A Comprehensive Measure of Well-Being
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/chetan926/HDI-Insight-.git
Programming Language(s)	Python (backend), TypeScript / JavaScript (frontend)
Framework(s) Used	Flask, Flask-CORS (backend REST API); React 19, Vite, Tailwind CSS, Recharts, Framer Motion (frontend)
Key Features Implemented	• REST API (Flask) with endpoints for health check, country list, HDI prediction, analytics, and model info • HDI prediction engine supporting multiple models: Linear Regression, Random Forest, Gradient Boosting, and Neural Network, plus a mathematical (UNDP formula) mode

	• Automatic computation of the three normalized UNDP sub-indices: Life Expectancy Index, Education Index, and Income Index, alongside the overall HDI score and category (Very High / High / Medium / Low) • Analytics dashboard data: average/highest/lowest HDI, HDI progression trend, radar chart of dimension balance, GNI-vs-life-expectancy scatter data, and category distribution counts • Model info endpoint exposing training pipeline steps, R², MAE, RMSE, and version/change logs • React + TypeScript frontend (Vite, Tailwind CSS) with Home, Dashboard, Predict, History, Model Info, About, and Not Found pages • Reusable UI components: AnimatedCounter, GlassCard, ProgressRing, StatCard, and layout components (Header, Footer, Background, Loader, Logo) • Interactive charts via Recharts, animations via Framer Motion, searchable/sortable/paginated prediction history page, CSV export and clearhistory actions • Form handling with react-hook-form, country dropdown with react-select, toast notifications with react-hot-toast • CORS-enabled API for local frontend/backend integration
Pending / Incomplete Features	• No automated test suite (unit/integration tests) currently included in the repository • No user authentication or role-based access control • Prediction history/count is kept in memory on the backend (predictions_run_counter) and is not persisted to a database, so it resets on server restart • No .env-based configuration; Kaggle dataset access and app settings are hardcoded in the source • Live end-to-end run (Kaggle dataset download via kagglehub) was not verified in this review session due to lack of internet access; please verify locally before submission
Setup / Run Instructions	Backend (Flask API): • cd backend • pip install -r requirements.txt • python app.py (starts the API on http://localhost:5000; first run downloads the Kaggle HDI dataset and trains the models) Frontend (React + Vite): • cd frontend • npm install • npm run dev (starts the dev server, default http://localhost:5173)

Code Quality Checklist

S.N o	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

This checklist was completed via static code review of the submitted repository (backend Flask API and React/TypeScript frontend). Backend modules are split by responsibility (app.py for routing, services/preprocessing.py, services/predictor.py, services/trainer.py); frontend code is organized into components, pages, hooks, api, and utils folders. Docstrings and inline comments are present in the backend services. Try/except blocks and input validation guard the prediction, analytics, and model-info endpoints.